

**UNIVERSIDAD TECNOLÓGICA NACIONAL**  
**Tecnicatura Universitaria en Programación - a distancia**

**TRABAJO PRÁCTICO INTEGRADOR**

***Gestión de Datos de Países en Python:***

*filtros, ordenamientos y estadísticas*

**ASIGNATURA:**

*Programación 1*



**INTEGRANTES**

MALDONADO, Georgina Daniela

TERMIGNONI, Ana

**PROFESORES**

**Profesor Coordinador:**

*Alberto Cortez*

**Profesor a cargo comisión:**

*Luciano Chirolí*

*Cinthia Rigoni*

*Martin Garcia*

*Ariel Enferrel*

**15 de junio de 2026**

## ÍNDICE

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>OBJETIVO DEL TRABAJO</b>                       | <b>3</b>  |
| <b>MARCO TEÓRICO E INVESTIGACIÓN</b>              | <b>4</b>  |
| Estructuras de Datos Lineales y Compuestas        | 4         |
| Funciones y Modularización                        | 5         |
| Estructuras de Control                            | 6         |
| Algoritmos de Ordenamiento                        | 7         |
| Persistencia de Datos y Archivos CSV              | 8         |
| <b>DISEÑO DEL CASO PRÁCTICO</b>                   | <b>9</b>  |
| Diagrama de Flujo                                 | 9         |
| Metodología                                       | 11        |
| <b>RESULTADOS OBTENIDOS Y EVIDENCIAS GRÁFICAS</b> | <b>12</b> |
| <b>LINKS AL REPOSITORIO Y VIDEO</b>               | <b>17</b> |
| <b>REFLEXIONES FINALES</b>                        | <b>18</b> |
| <b>FUENTES BIBLIOGRÁFICAS</b>                     | <b>19</b> |

## OBJETIVO DEL TRABAJO

El presente proyecto tiene como objetivo principal consolidar y aplicar los conocimientos fundamentales de la programación estructurada y modular adquiridos durante el cuatrimestre en la asignatura Programación 1. Mediante el diseño e implementación de un sistema de gestión de datos de países en consola, se ejercita la manipulación de colecciones dinámicas de datos, persistencia en archivos externos (CSV), el diseño de algoritmos de ordenamiento manual y la generación de métricas analíticas e indicadores estadísticos básicos.

Asimismo, el proyecto busca simular un entorno de desarrollo real donde se resuelva la problemática de la volatilidad de la memoria RAM a través de técnicas de persistencia de datos. De esta manera, el sistema no solo funciona como una herramienta de consulta interactiva para el usuario, sino que demuestra la capacidad de diseñar software eficiente, con un uso optimizado de los recursos de hardware, código limpio y un control estricto de errores y excepciones en tiempo de ejecución.

## MARCO TEÓRICO E INVESTIGACIÓN

### Estructuras de Datos Lineales y Compuestas

#### Listas y Diccionarios

En Python, el manejo eficiente de conjuntos de datos requiere de estructuras flexibles. Para este sistema se seleccionó una estructura combinada: una lista que aloja diccionarios en su interior.

- **Listas<sup>1</sup>**: Son colecciones de elementos ordenadas y mutables. Permiten almacenar de forma secuencial cada uno de los registros de los países para su posterior manipulación.
- **Diccionarios<sup>2</sup>**: Son estructuras asociativas basadas en parejas de "clave-valor". A diferencia de las listas, no acceden a sus elementos mediante un índice numérico, sino mediante claves únicas descriptivas.

En este proyecto, cada país se representa como un diccionario con las claves fijas: '**nombre**', '**poblacion**', '**superficie**' y '**continente**'. Esto dota al registro de una alta semántica y legibilidad en el código.

Esta implementación híbrida o compuesta (lista de diccionarios) permite unificar las ventajas de ambas estructuras. Al anidar cada diccionario dentro de una lista indexada, el sistema adquiere la capacidad de crecer de forma dinámica en la memoria intermedia a medida que el usuario registra nuevos países, eliminando las restricciones de tamaño fijo propias de los arreglos estáticos de otros lenguajes.

Asimismo, al iterar la lista mediante bucles, se puede acceder directamente a los atributos internos de cada país a través de sus claves de texto en un tiempo de procesamiento óptimo. Esta combinación resulta fundamental para el correcto funcionamiento de los módulos de filtrado y ordenamiento analítico del sistema, ya que provee una abstracción de datos limpia y ordenada que actúa como paso previo al almacenamiento físico en el archivo persistente.

## Funciones y Modularización

La modularización es una técnica de diseño de software que consiste en dividir un programa informático en partes independientes denominadas módulos o funciones. Cada función realiza una tarea única y específica, respetando el principio de responsabilidad única.

La división del sistema en módulos y funciones independientes (como la carga del archivo, el filtrado o el ordenamiento) disminuye la complejidad cognitiva del código, evita la duplicación de bloques de instrucciones y facilita el aislamiento y resolución de errores informáticos.

En el diseño de este sistema, la modularización se implementó mediante la fragmentación del código en módulos especializados:

- **utilidades.py** – Contiene la función `normalizar_texto()`, utilizada por otros módulos para eliminar acentos y convertir texto a minúsculas, garantizando búsquedas y comparaciones consistentes sin importar la capitalización o tildes.
- **database.py** – Agrupa las funciones de persistencia: `verificar_e_inicializar_csv()`, `cargar_desde_csv()` y `guardar_en_csv()`. Estas se encargan exclusivamente de la comunicación con el almacenamiento secundario (archivo `países.csv`).
- **operaciones.py** – Encapsula la lógica de negocio que modifica la lista en memoria RAM: `agregar_pais()` y `actualizar_pais()`. Estas funciones validan entradas, evitan duplicados y actualizan los datos del país.
- **reportes.py** – Reúne todas las funciones de consulta y presentación: `mostrar_tabla_paises()`, `buscar_pais_nombre()`, `filtrar_paises()`, `ordenar_paises()` y `mostrar_estadisticas()`. Cada una genera una salida distinta sin alterar los datos originales.
- **main.py** – Punto de entrada del programa. Contiene únicamente el menú interactivo y la función `ejecutar_menu_tpi()`, que coordina la llamada a los demás módulos.

Un aspecto técnico crucial de esta estructura modular es el aprovechamiento del pasaje de parámetros por referencia que realiza Python de forma nativa sobre los objetos mutables. Al transferir la lista central de países a funciones como `ordenar_paises()`, `actualizar_pais()` o `filtrar_paises()`, los cambios impactan directamente sobre la estructura original alojada en la memoria intermedia. Esto evita tener que duplicar variables globales o realizar copias innecesarias en el entorno local de cada módulo, garantizando un flujo de datos limpio, seguro y eficiente a lo largo de toda la ejecución del script.

## Estructuras de Control

- **Condicionales (if-elif-else):**

Actúan como bifurcaciones lógicas que permiten al programa tomar decisiones en tiempo de ejecución basándose en evaluaciones booleanas<sup>4</sup>. Son esenciales para validar las entradas del usuario (como evitar poblaciones negativas) y procesar las opciones del menú interactivo.

- **Repetitivas (for / while):**

El ciclo "while" mantiene el menú interactivo en consola activo de manera indefinida hasta que el usuario decida explícitamente cerrarlo seleccionando la opción de salida<sup>4</sup>. Por otro lado, el ciclo "for" se utiliza para iterar sobre las colecciones y procesar de forma uniforme cada país de la lista en las búsquedas, filtros y cálculos estadísticos.

La combinación e interconexión de ambas estructuras de control (anidamiento) constituye el motor algorítmico del sistema. Por ejemplo, al ejecutar un filtro analítico por continente o rango de habitantes, el bucle repetitivo se encarga de recorrer secuencialmente cada celda de la colección, mientras que un bloque condicional interno evalúa si las propiedades específicas del diccionario actual coinciden con los parámetros estipulados por el usuario. Este patrón de diseño permite filtrar y procesar grandes volúmenes de datos en una cantidad mínima de líneas de código, asegurando que el flujo del programa responda de manera exacta y predictiva ante cada interacción en la consola.

## Algoritmos de Ordenamiento

### Método de la Burbuja (Bubble Sort<sup>5</sup>)

El ordenamiento de datos es una operación fundamental en computación. Para este trabajo práctico se implementó de forma manual el algoritmo de Ordenamiento de Burbuja (Bubble Sort).

Su funcionamiento consiste en recorrer repetidamente la lista que se desea ordenar, comparando elementos adyacentes y mutando sus posiciones si se encuentran en el orden incorrecto. El proceso se repite iterativamente hasta que la lista queda completamente ordenada. Aunque presenta una complejidad temporal de  $O(n^2)$ , es ideal para propósitos pedagógicos debido a su simplicidad conceptual y su implementación nativa sin dependencias externas.

La función de ordenamiento desarrollada (**ordenar\_paises()**) adapta el algoritmo clásico de la burbuja para operar sobre colecciones de diccionarios. El algoritmo no solo compara valores primitivos, sino que accede a las claves numéricas ('poblacion' o 'superficie') o alfabéticas ('nombre') según el requerimiento del usuario. Además, se integró un parámetro de dirección lógico que altera el operador de comparación (**\$>\$ o \$<\$**), permitiendo alternar de forma dinámica entre un ordenamiento ascendente o descendente mediante el mismo bucle iterativo.

Dentro del desarrollo lógico de **ordenar\_paises()**, el proceso crítico de reordenamiento de los registros se ejecuta mediante la técnica de asignación simultánea o el uso de variables auxiliares de intercambio. Cuando la condición lógica determina que dos diccionarios contiguos están desordenados, el programa almacena temporalmente el objeto en una posición intermedia para evitar que se sobrescriba y se pierdan sus atributos durante la mutación de índices en la lista.

A pesar de que el algoritmo efectúa múltiples pasadas iterativas sobre la estructura de datos en memoria intermedia (RAM), esta implementación manual garantiza una fiabilidad absoluta del ordenamiento tabular antes de realizar cualquier renderizado visual en la consola o de salvar de forma permanente las modificaciones dentro del archivo de persistencia final.

## Persistencia de Datos y Archivos CSV

La memoria RAM de una computadora es volátil; todos los datos cargados en variables se pierden al finalizar la ejecución del script. Para lograr la persistencia de los datos entre sesiones, se recurre al almacenamiento secundario utilizando archivos de texto con formato CSV ("Comma-Separated Values"). El formato CSV organiza la información en filas, donde cada propiedad se separa mediante un delimitador. Python provee el módulo nativo "csv"<sup>6</sup>, que automatiza la lectura y escritura de estos archivos controlando las excepciones de formato, espacios innecesarios y garantizando la integridad de los datos.

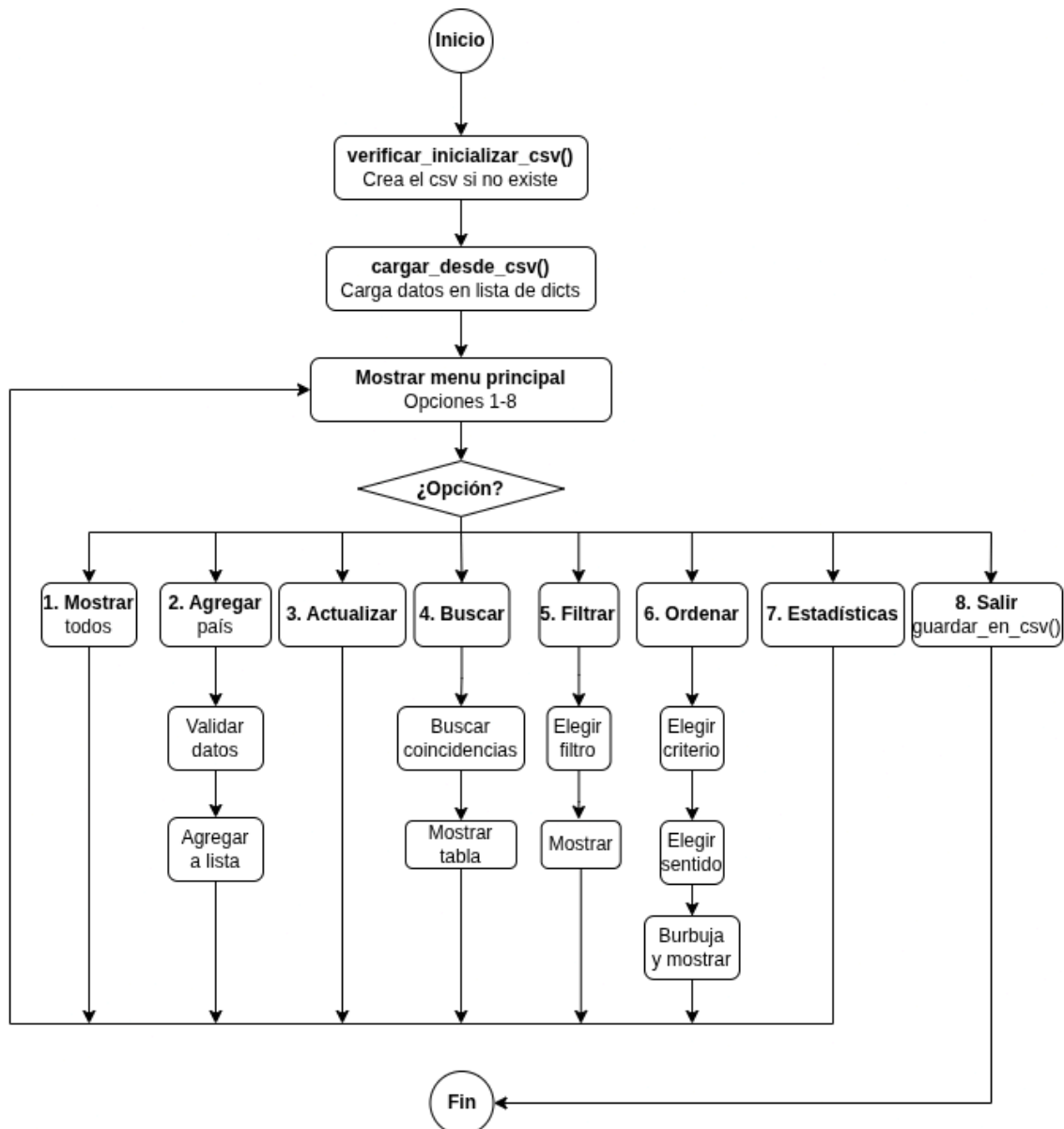
Para garantizar una ejecución fluida y evitar fallos por ausencia de recursos, el sistema implementa una subfunción de verificación inicializadora. Al arrancar el script, el programa comprueba la existencia física de `paises.csv`. En caso negativo, el módulo crea el archivo de forma automatizada e inyecta una estructura de datos base con registros iniciales, asegurando que las funciones de lectura y renderizado tabular cuenten siempre con información estructurada válida desde el primer momento de ejecución.

Este mecanismo de persistencia mitiga el riesgo de pérdida de información ante cierres inesperados de la terminal. Al interactuar con el archivo de manera exclusiva en momentos clave (lectura al inicio y escritura al final), el sistema optimiza los ciclos de lectura/escritura del disco rígido, extendiendo la vida útil del almacenamiento físico y agilizando la velocidad de respuesta de la aplicación en la consola de comandos.

## DISEÑO DEL CASO PRÁCTICO

### Diagrama de Flujo

Para definir el comportamiento lógico del sistema previo a la codificación, se diseñó el siguiente mapa de procesos que ilustra cómo se toman las decisiones desde que arranca el programa hasta que se guardan los datos de forma persistente:



El recorrido secuencial del diagrama de flujo expuesto anteriormente detalla la lógica de control que gobierna la aplicación. El sistema inicia con un nodo de arranque que deriva inmediatamente en la subfunción de verificación del almacenamiento secundario. Si el algoritmo comprueba la inexistencia del archivo "países.csv", se bifurca hacia un proceso automatizado de escritura que inyecta los registros base predefinidos por el equipo; en caso de que el archivo ya exista, el flujo continúa directamente hacia la lectura y carga de datos en la lista dinámica de la memoria RAM.

Una vez completada la inicialización de las colecciones, el programa entra en un bucle repetitivo condicionado (mientras la variable de control sea distinta de la opción de salida). Este ciclo renderiza el menú interactivo en la terminal y aguarda la entrada del operador. Cada una de las opciones numéricas (desde la búsqueda por nombre, filtros analíticos, ordenamientos por el método de la burbuja, hasta el cálculo de métricas estadísticas) representa un bloque de proceso secundario encapsulado. Al finalizar la ejecución de cualquiera de estas funciones periféricas, el flujo retorna de manera cíclica al menú principal. Únicamente cuando el usuario ingresa la opción correspondiente a la salida, el programa rompe el bucle, ejecuta el volcado final de datos sobre el archivo CSV para asegurar la persistencia y alcanza el nodo de finalización del sistema.

## Metodología

El sistema fue concebido bajo el paradigma de programación estructurada. Previo a la codificación, se estableció un diseño conceptual fragmentado en módulos lógicos para evitar el acoplamiento de funciones.

La metodología de trabajo en equipo consistió en la división de tareas lógicas y unificación de código mediante pruebas de integración. Se optó por centralizar las operaciones mutables en una lista en memoria intermedia, postergando el guardado físico en el almacenamiento persistente ("países.csv") únicamente para cuando el usuario seleccione explícitamente la opción de salida del sistema, previniendo así operaciones innecesarias de escritura en el disco.

La arquitectura del software se estructuró de manera modular a través de una función principal llamada `ejecutar_menu_tpi()`, la cual actúa como el nodo central del sistema distribuido. Esta función transfiere por referencia la lista dinámica `países_sistema` hacia los módulos periféricos especializados de filtrado, búsqueda y ordenación algorítmica, garantizando que todas las operaciones se realicen en tiempo real sobre el mismo conjunto de datos.

Este enfoque metodológico basado en la cohesión modular asegura que cualquier modificación futura en la lógica de negocio (por ejemplo, añadir un nuevo campo como "Capital" a los países) requiera cambios mínimos en el código fuente. La separación de responsabilidades entre la interfaz de usuario en consola, el motor analítico de datos y el controlador del almacenamiento en disco constituye una excelente práctica de desarrollo alineada con los estándares de ingeniería de software requeridos por la cátedra.

## RESULTADOS OBTENIDOS Y EVIDENCIAS GRÁFICAS

Se logró una aplicación de consola sólida y robusta que cumple con la totalidad de los requerimientos de la cátedra. El programa responde adecuadamente ante entradas de datos erróneas (por ejemplo, si el usuario escribe letras en lugar de números para la población o ingresa rangos lógicos imposibles como una población mínima mayor a la máxima), informando con claridad los errores sin interrumpir intempestivamente la ejecución del programa. La función de normalización de nombres se aplica de manera sistemática a los nombres de países y continentes, permitiendo ignorar acentuación gráfica y diferencias entre mayúsculas y minúsculas al realizar las búsquedas y filtrados, y posibilitando de ese modo la consistencia de los resultados.

Para garantizar este nivel de estabilidad y robustez en la experiencia del operador, el código implementa bloques de control de excepciones "try-except".

Específicamente, el sistema mitiga fallos críticos mediante las siguientes validaciones en tiempo de ejecución:

1. **Control de Datos Alfabéticos (ValueError):** Al momento de agregar o actualizar registros, si el usuario introduce letras en los campos destinados a variables cuantitativas (como la población o la superficie), el programa captura la excepción, limpia el flujo de entrada y solicita los datos nuevamente sin corromper la lista en memoria RAM.
2. **Validación de Rangos Lógicos:** Las funciones de filtrado analítico comprueban que los límites numéricos mínimos ingresados no superen a los máximos, impidiendo cálculos con inconsistencias matemáticas.
3. **Gestión de Permisos sobre Archivos (PermissionError):** El software previene bloqueos del sistema operativo alertando al usuario mediante un mensaje en consola si el archivo de almacenamiento externo ("países.csv") se encuentra abierto de manera simultánea en otra aplicación (como Microsoft Excel), protegiendo la integridad de la persistencia de datos.

## Anexo: Evidencias Gráficas de la Ejecución en Consola

A continuación, se presentan las capturas de pantalla que certifican el correcto funcionamiento del sistema ante los diferentes flujos solicitados por la cátedra:

- **Captura 1:** Inicialización Automática del Dataset y Menú Principal.

```
TPI_programacion_I on 1 main via 2 v3.14.5
> python3 programa.py

=====
TPI PROGRAMACIÓN 1: GESTIÓN DE PAÍSES
=====
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar un país por nombre
5. Filtrar países (Continente / Rangos)
6. Ordenar países
7. Mostrar indicadores estadísticos
8. Guardar cambios y Salir
=====
Seleccione una opción (1-8): 1

-----
Nombre | Población | Superficie (km2) | Continente
-----
Argentina | 45376763 | 2780400 | America
Japón | 125800000 | 377975 | Asia
Brasil | 213993437 | 8515767 | America
Alemania | 83149300 | 357022 | Europa
Nauru | 11947 | 20 | Oceania
Vanuatu | 327778 | 12189 | Oceania
Australia | 26713205 | 7741220 | Oceania
Mongolia | 3493629 | 1564116 | Asia
Vietnam | 101343800 | 331210 | Asia
China | 1419321277 | 9598089 | Asia
Rusia | 143957079 | 17098242 | Europa
Zimbabwe | 15993524 | 390760 | Africa
Egipto | 106600000 | 1002450 | Africa
Canadá | 41288599 | 9984670 | America
Bosnia y Herzegovina | 3406568 | 51197 | Europa
Grecia | 10372000 | 131957 | Europa
Cuba | 10892659 | 110860 | America
Nigeria | 227882845 | 910770 | Africa
Seychelles | 99202 | 455 | Africa
-----
```

- **Captura 2:** Procesamiento del Algoritmo de Ordenamiento Manual (Bubble Sort).

Ordenamiento por superficie de forma ascendente

```
6. Ordenar países
7. Mostrar indicadores estadísticos
8. Guardar cambios y Salir
```

```
=====
Seleccione una opción (1-8): 6
```

```
=== OPCIONES DE ORDENAMIENTO ===
```

```
1. Ordenar por Nombre
2. Ordenar por Población
3. Ordenar por Superficie
Seleccione criterio (1-3): 3
1. Ascendente
2. Descendente
Seleccione sentido (1-2): 1
```

| Nombre               | Población  | Superficie (km2) | Continente |
|----------------------|------------|------------------|------------|
| Nauru                | 11947      | 20               | Oceania    |
| Seychelles           | 99202      | 455              | Africa     |
| Vanuatu              | 327778     | 12189            | Oceania    |
| Bosnia y Herzegovina | 3406568    | 51197            | Europa     |
| Cuba                 | 10892659   | 110860           | America    |
| Grecia               | 10372000   | 131957           | Europa     |
| Vietnam              | 101343800  | 331210           | Asia       |
| Alemania             | 83149300   | 357022           | Europa     |
| Japón                | 125800000  | 377975           | Asia       |
| Zimbabwe             | 15993524   | 390760           | Africa     |
| Nigeria              | 227882845  | 910770           | Africa     |
| Egipto               | 106600000  | 1002450          | Africa     |
| Mongolia             | 3493629    | 1564116          | Asia       |
| Argentina            | 45376763   | 2780400          | America    |
| Australia            | 26713205   | 7741220          | Oceania    |
| Brasil               | 213993437  | 8515767          | America    |
| China                | 1419321277 | 9598089          | Asia       |
| Canadá               | 41288599   | 9984670          | America    |
| Rusia                | 143957079  | 17098242         | Europa     |

- **Captura 3:** Reporte de Indicadores y Métricas Estadísticas.

```
=====
TPI PROGRAMACIÓN 1: GESTIÓN DE PAÍSES
=====
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar un país por nombre
5. Filtrar países (Continente / Rangos)
6. Ordenar países
7. Mostrar indicadores estadísticos
8. Guardar cambios y Salir
=====
Seleccione una opción (1-8): 7

=====
ESTADÍSTICAS GENERALES
=====
País con MAYOR población: China (1419321277 hab.)
País con MENOR población: Nauru (11947 hab.)
Promedio de población global: 135790716.42 habitantes
Promedio de superficie global: 3208387.84 km2

Cantidad de países por continente:
-> America: 4
-> Asia: 4
-> Europa: 4
-> Oceanía: 3
-> Africa: 4
=====
```

- **Captura 4:** Intercepción de Errores y Robustez del Sistema.

```
TPI_programacion_I on 1 main via 2 v3.14.5
> python3 programa.py

=====
TPI PROGRAMACIÓN 1: GESTIÓN DE PAÍSES
=====
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar un país por nombre
5. Filtrar países (Continente / Rangos)
6. Ordenar países
7. Mostrar indicadores estadísticos
8. Guardar cambios y Salir
=====
Seleccione una opción (1-8): 2

--- AGREGAR NUEVO PAÍS ---
Nombre del país: Finlandia
Población (entero): kjk
Error: La población y la superficie deben ser números enteros.
=====

TPI PROGRAMACIÓN 1: GESTIÓN DE PAÍSES
=====
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar un país por nombre
5. Filtrar países (Continente / Rangos)
6. Ordenar países
7. Mostrar indicadores estadísticos
8. Guardar cambios y Salir
=====
Seleccione una opción (1-8): █
```

## LINKS AL REPOSITORIO Y VIDEO

**Repositorio de GitHub:** [https://github.com/avavassa/TPI\\_programacion\\_l.git](https://github.com/avavassa/TPI_programacion_l.git)

**Video explicativo en Google Drive:**

 TPI\_Prog\_1\_Maldonado\_Termignoni.mp4

## REFLEXIONES FINALES

El desarrollo de este Trabajo Práctico Integrador nos permitió materializar los conceptos abstractos estudiados en las clases teóricas en una herramienta práctica y funcional. Comprendimos el valor real de la modularización al ver cómo un bloque complejo de lógica se simplifica al fragmentarse en funciones más pequeñas. Asimismo, la experiencia de interactuar con archivos externos nos brindó una perspectiva real de cómo los sistemas de software modernos manejan la información de manera persistente en entornos reales.

Por otra parte, la implementación manual del algoritmo de ordenamiento por el método de la burbuja y el desafío de adaptarlo a estructuras compuestas como listas de diccionarios fortaleció nuestra comprensión sobre la lógica algorítmica y la optimización de procesos en memoria. Este ejercicio nos demostró que el éxito de un sistema interactivo no radica únicamente en que cumpla con las tareas solicitadas, sino en la eficiencia interna con la que procesa la información antes de mostrarla en la interfaz del usuario.

Finalmente, el desarrollo de las rutinas de validación y normalización de entradas y el control preventivo de excepciones nos concientizó sobre la importancia de la robustez en el software. Entendimos que un buen diseño debe anticiparse siempre a los posibles errores del operador humano para evitar fallos críticos del sistema en tiempo de ejecución o resultados inconsistentes. Esta experiencia integradora consolida una base técnica sólida y fundamental en nuestra formación, preparándonos con herramientas y criterios reales para afrontar proyectos informáticos de mayor complejidad en las asignaturas venideras.

## FUENTES BIBLIOGRÁFICAS

1. Apuntes de Cátedra. (2025). Unidad 5: Listas. Universidad Tecnológica Nacional (UTN)
2. Apuntes de Cátedra. (2026). Unidad 7: Diccionarios. Universidad Tecnológica Nacional (UTN)
3. Apuntes de Cátedra. (2026). Unidad 6: Funciones. Universidad Tecnológica Nacional (UTN)
4. Downey, A. (2015). *Think Python: How to Think Like a Computer Scientist* (2nd ed., Version 2.4.0). Green Tea Press, Needham, Massachusetts.
5. Apuntes de Cátedra. (2025). Unidad 7: Estructuras de Datos y Algoritmos de Ordenamiento. Universidad Tecnológica Nacional (UTN)
6. Python Software Foundation. (2026). The Python Standard Library - csv File Reading and Writing. Recuperado de <https://docs.python.org/3/library/csv.html>
7. Van Rossum, G., & Drake, F. L. (2009). *An Introduction to Python*. Network Theory Ltd.